LSTM и GRU

Теоретический семинар

Статьи к обсуждению

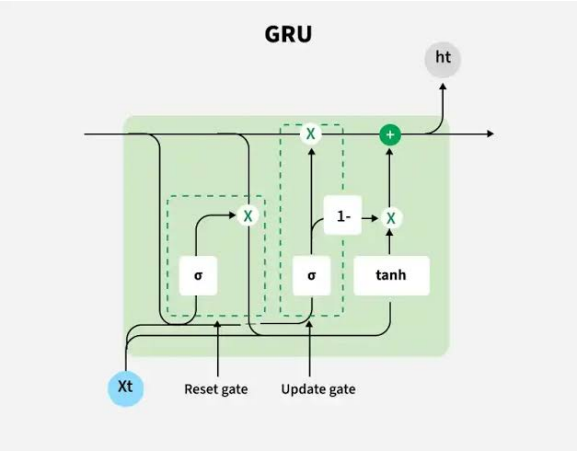
Архитектура	**Оригинальная работа**	**Год**
LSTM	Hochreiter & Schmidhuber, *Long Short-Term Memory*, Neural Computation	1997
GRU	Cho et al., *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation*	2014
Оригинальная архитектура	Hochreiter & Schmidhuber, *LSTM can solve hard long time lag problems*, NIPS	1996

О GRU

GRU была предложена как гейтированная скрытая единица внутри архитектуры encoder–decoder для нейронного машинного перевода.

Encoder–decoder

Схема из двух сетей: одна сжимает вход в вектор, другая разворачивает его в выход.


GRU
ht
X
+
X
1-
X
σ
σ
tanh
Xt
Reset gate
Update gate

Reset Gate

$$r_t = \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r)$$

Пояснение:

- X_t — входной вектор на шаге t.
- H_{t-1} — предыдущее скрытое состояние.
- W_{xr}, W_{hr} — матрицы весов, b_r — смещение.
- σ — сигмоида, сжимает результат в $(0, 1)$.
- r_t — вектор той же размерности, что и H_{t-1}: каждая компонента задаёт, какую долю соответствующей компоненты прошлого состояния сохранить.

Роль: определяет, какую часть предыдущего скрытого состояния забыть или проигнорировать, позволяя сети отбросить нерелевантный исторический контекст при появлении новых паттернов.

Update Gate

$$z_t = \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z)$$

Пояснение:

- X_t — входной вектор на шаге t.
- H_{t-1} — предыдущее скрытое состояние.
- W_{xz}, W_{hz} — матрицы весов, b_z — смещение.
- σ — сигмоида, сжимает результат в $(0, 1)$.
- z_t — вектор той же размерности, что и H_{t-1}: каждая компонента задаёт баланс между старым состоянием и кандидатом.

Роль: решает, какую часть прошлого скрытого состояния H_{t-1} перенести в текущий шаг, а какую заменить новой кандидатной информацией. Контролирует долговременные зависимости.

Candidate Hidden State

$$\tilde{H}_t = \tanh(X_t W_{xh} + (r_t \odot H_{t-1})\, W_{hh} + b_h)$$

Пояснение:

- X_t — входной вектор на шаге t.
- $r_t \odot H_{t-1}$ — прошлое состояние, промодулированное reset-гейтом: нерелевантные компоненты гасятся.
- W_{xh}, W_{hh} — матрицы весов, b_h — смещение.
- tanh — гиперболический тангенс, сжимает результат в $(-1, 1)$.
- $\tilde{H}_t$ — предварительное новое состояние памяти.

Роль: prospective new memory state, вычисляемое по текущему входу и прошлому скрытому состоянию, промодулированному reset-гейтом.

Final Hidden State

$$H_t = z_t \odot H_{t-1} + (1 - z_t) \odot \tilde{H}_t$$

Пояснение:

- z_t — update-гейт, $\tilde{H}_t$ — кандидатное состояние.
- $z_t \odot H_{t-1}$ — доля старого состояния, которую сохраняем.
- $(1 - z_t) \odot \tilde{H}_t$ — доля кандидата, которую принимаем.
- $\odot$ — поэлементное умножение (Hadamard product).

Роль: итоговый выход текущего шага, комбинирующий старое скрытое состояние и кандидатное через update-гейт.

Вопросы по статье про GRU (1/2)

Q1. Зачем нужен гейт сброса?

Q2. Чем контекстный вектор c отличается от скрытого состояния h_t?

Вопросы по статье про GRU (2/2)

Q3. Что авторы визуализируют и зачем?

Статья по LSTM

Hochreiter, S., & Schmidhuber, J. (1997).
Long Short-Term Memory.
Neural Computation, 9(8), 1735–1780.

LSTM: проблема

- Стандартная RNN не справляется с длительными временными задержками (long time lag).
- При обратном распространении ошибки градиент затухает.
- Сигнал старше примерно 5–10 шагов практически теряется.

Long time lag

Ситуация, когда нужный сигнал отделён от момента его использования многими шагами.

LSTM: решение

- Поток ошибки с постоянной скоростью (constant error flow).
- Гейтовые блоки.
- «Лента конвейера ошибки» (constant error carousel) позволяет информации долго сохраняться без затухания.

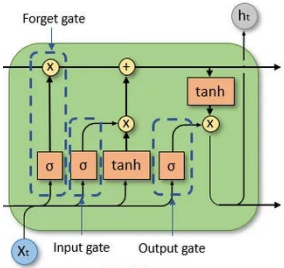
Гейт

Обучаемый фильтр, который решает, сколько информации пропустить.

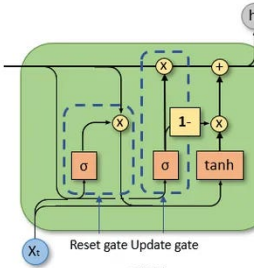
Предшествующая работа по LSTM

Hochreiter, S., & Schmidhuber, J. (1996).
LSTM can solve hard long time lag problems.
NIPS 9.

Предварительная версия журнальной статьи 1997 года. Демонстрирует экспериментальное сравнение: LSTM решает задачи с длительными задержками, которые предыдущие алгоритмы на RNN не могли решить.


Forget gate
h_t
tanh
σ
σ
tanh
σ
X_t
Input gate
Output gate

LSTM


1-
σ
σ
tanh
X_t
Reset gate
Update gate

GRU

LSTM: два информационных потока

Ячейка LSTM поддерживает два различных состояния, передаваемых между шагами:

- **Cell State** (C_t) — «конвейерная лента» или долговременная память. Идёт вдоль всей цепочки с минимальными линейными взаимодействиями, за счёт чего градиент течёт обратно без затухания.
- **Hidden State** (h_t) — кратковременная память. Влияет на выход текущего шага и передаётся в следующий модуль.

LSTM: гейтовые механизмы

Поток информации регулируется тремя сигмоидными гейтами. Выход каждого — от 0 (полностью блокировать) до 1 (полностью пропустить):

- **Forget Gate** (f_t) — какую долю информации отбросить из предыдущего состояния C_{t-1}.
- **Input Gate** (i_t и $\tilde{C}_t$) — какую новую информацию из x_t и h_{t-1} добавить в состояние ячейки.
- **Output Gate** (o_t) — какую часть обновлённого C_t выдать в новое кратковременное состояние h_t.

Forget Gate

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Пояснение:

- h_{t-1} — предыдущее скрытое состояние, x_t — текущий вход.
- $[h_{t-1}, x_t]$ — конкатенация векторов.
- W_f, b_f — обучаемые веса и смещение.
- σ — сигмоида, сжимает результат в $(0, 1)$.
- f_t — вектор той же размерности, что C_{t-1}: каждая компонента задаёт, какую долю соответствующей ячейки памяти сохранить.

Роль: решает, какую часть предыдущего состояния ячейки C_{t-1} отбросить.

Input Gate и кандидатное состояние

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

Пояснение:

- i_t — сигмоидный гейт: какие компоненты кандидата принять.
- $\tilde{C}_t$ — кандидатное состояние: новая информация, полученная из входа и прошлого скрытого состояния.
- tanh сжимает кандидата в $(-1, 1)$.

Роль: определяет, какую новую информацию из x_t и h_{t-1} следует добавить в состояние ячейки.

Обновление состояния ячейки

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Пояснение:

- $f_t \odot C_{t-1}$ — доля старой памяти, которую сохраняем.
- $i_t \odot \tilde{C}_t$ — доля новой информации, которую записываем.
- $\odot$ — поэлементное умножение.

Роль: итоговое обновление долговременной памяти ячейки.

Output Gate и обновление скрытого состояния

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

Пояснение:

- o_t — сигмоидный гейт: какие компоненты обновлённого C_t выдать наружу.
- $\tanh(C_t)$ — нормализация состояния ячейки в $(-1, 1)$.
- h_t — новое кратковременное состояние, которое идёт на выход и в следующий шаг.

Роль: контролирует, какая часть обновлённой памяти станет новым скрытым состоянием.

Обозначения

- x_t — входной вектор на шаге t.
- h_{t-1}, h_t — предыдущее и текущее скрытые состояния (кратковременная память).
- C_{t-1}, C_t — предыдущее и текущее состояния ячейки (долговременная память).
- W, b — обучаемые матрицы весов и векторы смещений для соответствующих гейтов.
- σ — сигмоида, tanh — гиперболический тангенс.
- $\odot$ — поэлементное (Hadamard) произведение векторов.

Архитектурные варианты

- **Bidirectional LSTM (BiLSTM)** — обрабатывает последовательность в прямом и обратном направлениях, одновременно учитывая прошлый и будущий контекст.
- **Stacked LSTM** — многослойная конфигурация, позволяющая извлекать иерархические признаки из сложных временных рядов и текстов.
- **Peephole Connections** — гейты получают прямой доступ к состоянию ячейки C_t, что позволяет точнее выучивать временные зависимости.
- **GRU** — упрощённая альтернатива: объединяет состояние ячейки и скрытое состояние, использует два гейта вместо трёх и обучается быстрее.

Вопросы по статье Шмидхубера

Q1. Какую именно проблему решает LSTM? Сформулируйте одним предложением.

Q2. Что такое «constant error carousel» и зачем он нужен?

Q3. Какие задачи использовались для проверки? Приведите пример.

Вопросы по сравнению LSTM и GRU

Q1. Сколько гейтов у каждого и в чём разница?

Q2. Что передаётся между шагами в LSTM и что в GRU?

Q3. Когда GRU предпочтительнее, а когда LSTM?